

Urban Infrastructure AI Monitoring and Resilience Analysis

Complete beginner guide for understanding the project from 0 to 100 percent.

Prepared for someone who is new to the codebase, new to the research pipeline, and possibly new to Python infrastructure projects.

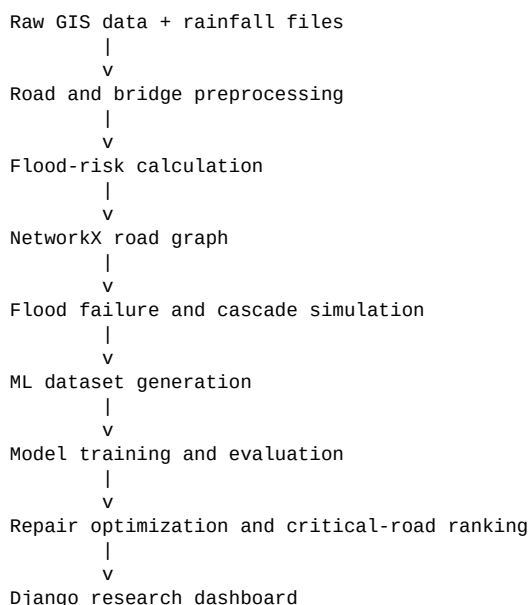
1. What This Project Is

This project is a research-style prototype for studying how an urban road network can break down during flooding.

The project uses Pune road-network data, rainfall-derived flood-risk signals, graph simulation, machine learning, repair-strategy optimization, and a Django dashboard. In simple words, it asks:

- Which roads are exposed to rainfall or flood risk?
- If some roads fail first, can that failure spread to nearby roads?
- Can a machine-learning model predict which roads will fail?
- Which roads should be protected or repaired first when the budget is limited?
- Can all of this be shown in a dashboard for demo, viva, or report use?

The project is not only a website. The website is the final surface. Behind it is a full pipeline:



2. The Problem Being Solved

Cities depend on roads, bridges, and junctions. During heavy rainfall or flooding, a few weak or exposed road segments may become unusable. That can overload nearby roads, force traffic to reroute, and create a wider failure

pattern.

This project models that idea computationally. It treats roads as connected parts of a graph. Every road can have attributes such as traffic, length, rainfall, flood risk, capacity, stress, and failure state. Then the system simulates what happens when rainfall causes some roads to fail.

The project is useful as a final-year or research prototype because it combines:

- geospatial data processing
- rainfall and flood-risk scoring
- graph algorithms
- cascading failure simulation
- tabular machine learning
- graph neural-network experiments
- temporal failure prediction
- optimization under repair budgets
- dashboard presentation

3. Important Beginner Concepts

GIS

GIS means Geographic Information System. In this project, roads and bridges are stored as geographic shapes. The main format is GeoJSON.

A road is usually a `LineString` or `MultiLineString`, which means it is drawn as one or more lines on a map.

CRS

CRS means Coordinate Reference System. A common map CRS is `EPSG:4326`, which stores latitude and longitude. For measuring distance in meters, the notebooks project the data to a local metric CRS such as `EPSG:32643`.

Graph

A graph is a structure made of nodes and edges.

In this project:

- a road segment is represented as a road node
- graph edges connect roads that touch or are considered neighbors
- graph attributes store road details such as traffic and flood risk

Cascade

A cascade is a chain reaction. If a road fails, nearby roads may receive extra stress. If that stress becomes too high, they may fail too. This produces waves of failure across the network.

ML Dataset

The machine-learning dataset is a table where each row represents a road in a simulated scenario. The model learns from road features and tries to predict whether the road failed.

Django

Django is the web framework used for the dashboard. It handles URLs, database models, templates, pages, admin screens, and management commands.

4. Repository Layout

EDI/	
data/	
gis/	GeoJSON road, bridge, and boundary data
graphs/	NetworkX graph files and simulation outputs
weather/	raw rainfall NetCDF files
dashboard_artifacts/	precomputed dashboard JSON files
data_ingestion/	rainfall loading and preview utilities
docs/	project notes and this guide
graph/	cascade simulation logic
infra_ai_system/	Django project and dashboard app
ml/	dataset generation and model training
notebooks/	exploratory GIS and graph-building notebooks
optimization/	repair and budget strategy scripts
risk/	rainfall-to-flood-risk logic
static/	static data copies for web use
visualization/	plotting and map-generation scripts
run_scenario.py	main standalone cascade simulation script
requirements.txt	Python dependencies

5. Main Technologies

- pandas: table processing
- geopandas: GIS files and road geometry
- xarray: rainfall NetCDF files
- shapely: geometric operations
- networkx: graph representation and graph algorithms
- scikit-learn: tabular ML models and evaluation
- torch: neural networks
- torch-geometric: graph neural networks
- matplotlib and imageio: plots and animations
- Django: research dashboard
- SQLite: local dashboard database
- ReportLab: this PDF generator

6. Data Flow In Plain English

The project begins with raw geographic road data and rainfall data. The notebooks clean and clip the GIS data to Pune, assign road IDs, calculate length and traffic-like scores, and build a graph. The rainfall module samples rainfall near each road and converts it into a flood-risk score.

After that, scenario scripts simulate disasters. A storm severity scale controls how likely roads are to fail initially. Failed roads then stress their neighbors. The cascade function repeats until no new road failures appear or a maximum step count is reached.

Those simulation outputs are saved. They become:

- graph pickle files for later analysis
- JSON files for cascade summaries
- CSV files for ML datasets
- trained model pickle files
- metrics JSON files
- dashboard-ready JSON artifacts

Finally, the Django app reads the database and saved artifacts to show the scenario, maps, model results, optimization results, and route planning.

7. Building The Project From Scratch

This section explains how the project could be rebuilt from an empty folder.

Step 1: Create The Project Structure

Create folders for data, graph logic, risk logic, ML logic, optimization, visualization, notebooks, docs, and the Django app.

```
EDI/  
  data/  
  graph/  
  risk/  
  ml/  
  optimization/  
  visualization/  
  notebooks/  
  infra_ai_system/
```

Step 2: Install Python Packages

The dependency list is in `requirements.txt`.

Typical setup:

```
python -m venv .venv  
.\venv\Scripts\activate  
pip install -r requirements.txt
```

Step 3: Prepare GIS Data

Use ``notebooks/01_prepare_gis.ipynb`` to:

- load Pune boundary data
- load raw roads
- load raw bridges
- clip roads and bridges to Pune
- assign IDs such as ``road_0``, ``road_1``, and so on
- save ``roads_final.geojson`` and ``bridges_final.geojson``

Use ``notebooks/02_load_gis.ipynb`` to:

- inspect GIS columns
- project roads into a meter-based CRS
- calculate road lengths
- assign traffic weights using the road type
- connect roads and bridges
- build and save the NetworkX graph

Step 4: Attach Rainfall And Flood Risk

Use ``risk/flood_risk.py``.

The script:

- finds a rainfall NetCDF file for a year
- clips rainfall to a Pune bounding box
- loads final road geometry
- picks a midpoint for each road
- samples rainfall at the nearest rainfall grid cell
- calculates mean rainfall
- calculates a flood-risk score between 0 and 1
- returns roads with ``rain_mm_mean`` and ``flood_risk``

Step 5: Build The Base Graph

The base graph is stored at:

```
data/graphs/pune_base_graph_weather.pickle
```

It is a Python pickle containing a NetworkX graph. Road nodes contain attributes such as:

- ``type``
- ``traffic``

- `length`
- `rain\_mm\_mean`
- `flood\_risk`
- `failed`
- `capacity`
- `base\_traffic`
- `stress`

Step 6: Run A Flood Scenario

Use `run\_scenario.py`.

The script loads the base graph, sets scenario parameters, seeds initial failures from flood risk, runs the cascade simulation, and saves results.

Important severity examples:

- `0.7`: mild scenario
- `1.5`: severe scenario
- `2.5`: extreme scenario

Output examples:

- `data/graphs/run\_results\_weather\_0.7.json`
- `data/graphs/run\_results\_weather\_1.5.json`
- `data/graphs/run\_results\_weather\_2.5.json`
- `data/graphs/pune\_after\_cascade\_weather\_0.7.gpickle`

Step 7: Generate Machine-Learning Datasets

Use:

```
python -m ml.dataset
python -m ml.temporal_dataset
```

The tabular dataset saves to:

```
data/ml_dataset.csv
```

The temporal dataset saves to:

```
data/temporal_dataset.csv
```

Step 8: Train Models

The strongest current training script is:

```
python -m ml.train_model
```

It compares:

- Random Forest
- Extra Trees
- Logistic Regression

It saves:

- best model pickle
- model metrics JSON
- model comparison JSON
- feature importance CSV

Experimental neural-network scripts also exist:

- ``ml/train_nn.py``
- ``ml/train_temporal.py``
- ``ml/train_gnn.py``
- ``ml/train_gnn_generalization.py``
- ``ml/train_stgmn.py``

Step 9: Run Optimization

Use:

```
python -m optimization.compare_strategies
```

This compares greedy repair against random repair for different repair budgets.

Greedy repair uses ``optimization/greedy_repair.py``, which ranks roads by a combined normalized score of degree centrality and flood risk.

Step 10: Prepare The Dashboard

From ``infra_ai_system/``:

```
python manage.py migrate
python manage.py ingest_research_data
python manage.py export_dashboard_artifacts
python manage.py runserver
```

Then open:

```
http://127.0.0.1:8000/
```

8. Core Algorithm: Flood Failure Seeding

The first failure stage is probabilistic. For every road, the code looks at ``flood_risk``. Higher risk means a higher chance that the road fails at the start of the scenario.

Plain-English logic:

```
for every road:
    risk = road flood risk between 0 and 1
    probability = risk multiplied by scenario scale
    draw a random number between 0 and 1
    if random number is smaller than probability:
        mark road as failed
```

This is why scale matters. A scale of 0.7 creates fewer initial failures. A scale of 2.5 creates many more.

9. Core Algorithm: Cascade Simulation

The main cascade function lives in `graph/cascade.py`.

The idea is:

```
find roads that already failed
for each road that is still working:
    count how many failed road neighbors it has
    calculate extra overload caused by those failed neighbors
    add overload/capacity to the road stress value
    if stress is greater than 1:
        convert stress into a failure probability
        maybe mark the road as failed
return the list of newly failed roads
```

Key variables:

- `base\_traffic`: original traffic value for the road
- `capacity`: estimated road tolerance before failure
- `overload\_factor`: how strongly neighbor failures increase stress
- `stress`: accumulated pressure over cascade steps
- `failed\_neighbors`: nearby failed road nodes

This is a simple research heuristic, not a real hydraulic or traffic-engineering simulator. Its value is that it gives controlled failure labels and lets the project study network behavior.

10. Scenario Tuning

`run\_scenario.py` adjusts capacity and overload settings based on severity.

Mild scenarios:

- higher capacity factor
- lower overload factor
- fewer cascades

Severe or extreme scenarios:

- lower capacity factor

- higher overload factor
- larger cascades

The current tuning is:

```
scale <= 0.8:
  capacity factor = 1.5
  overload factor = 1.2

scale <= 1.6:
  capacity factor = 1.25
  overload factor = 1.5

scale > 1.6:
  capacity factor = 1.1
  overload factor = 1.8
```

11. Machine Learning Explained

The ML part tries to predict road failure from features.

Each row in `data/ml\_dataset.csv` is one road under one simulated scenario. The target column is `failed`.

Important feature groups:

- physical or assigned road features: traffic, length
- rainfall features: rain, flood risk
- graph features: degree, clustering, core number, PageRank
- neighbor features: neighbor risk, neighbor traffic, neighbor degree
- failure context: failed neighbors and failed-neighbor ratio
- scenario context: scale and exposure score

The best tabular training script filters out roads that were initially failed, so the model focuses on predicting cascade-driven failures among roads that were alive at the start.

The train/validation/test split is grouped by `simulation\_id`. This is important because rows from the same simulation are related. Grouped splitting reduces leakage between train and test sets.

12. Dashboard Explained

The dashboard is a Django app inside:

```
infra_ai_system/dashboard/
```

The key pages are:

- `/`: dashboard overview with map, scenario selector, severity selector, cascade summary, training snapshot, and optimization snapshot
- `/scenarios/`: scenario explorer with severity comparison chart and cascade interpretation
- `/models/`: model results page with model metrics, candidate comparison, and feature importance

- `/optimization/`: repair optimization page with greedy-vs-random budget comparison
- `/routing/`: safe rerouting planner
- `/routing/search/`: JSON endpoint for road search
- `/runs/<slug>/`: experiment run detail page

The dashboard uses saved artifacts for speed. Instead of enriching every road on every request, `export_dashboard_artifacts` writes precomputed JSON files into `data/dashboard_artifacts/`.

13. Django Database Model

Django stores structured experiment information using these models:

- `ScenarioConfig`: a scenario definition such as Pune extreme flood 2020
- `ExperimentRun`: one run of a cascade, dataset build, training job, optimization job, or dashboard snapshot
- `RunArtifact`: a file produced by a run, such as a graph, CSV, JSON, model, plot, or report
- `ModelResult`: model metrics and metadata
- `OptimizationResult`: repair-strategy results for different budgets

This design is useful because it turns loose files into explainable experiment records.

14. Safe Rerouting Feature

The routing page lets a user choose a source road and target road. It then computes a route while avoiding roads that failed under the selected severity.

The route cost changes by mode:

- `safe`: strongly penalizes flood risk and criticality
- `balanced`: trades safety against route burden
- `fast`: leans toward shorter and lighter-traffic paths
- `emergency`: strongly avoids risky and critical segments

Internally it uses NetworkX shortest path with custom edge weights.

15. Outputs And Artifacts

Important output types:

- `.geojson`: geographic road and bridge data
- `.gpickle`: saved NetworkX graphs
- `.json`: simulation summaries, dashboard artifacts, model metrics
- `.csv`: ML datasets, critical-road rankings, feature importance
- `.pkl`: trained model files

- `.png` or `.gif``: visualization outputs

You should think of the repo as having two layers:

- source code that knows how to generate results
- generated artifacts that store the results for analysis and dashboard use

16. How To Explain This Project In A Viva

A strong short explanation:

This project builds a flood-resilience analysis system for Pune road infrastructure. It converts GIS road data and rainfall data into a graph where each road has traffic, length, rainfall, and flood-risk attributes. A flood scenario first fails roads probabilistically based on flood risk, then a cascade model spreads failures through neighboring roads based on overload and capacity. The resulting simulations are used to train ML models that predict road failure and to compare repair strategies under limited budgets. A Django dashboard presents the scenario maps, cascade metrics, model results, optimization results, and safe rerouting.

17. Limitations To State Honestly

- Flood risk is rainfall-derived and spatially coarse.
- Failure labels are simulation-generated, not real observed road failure labels.
- Very high ML scores may reflect the structure of the simulation labels.
- Critical-road rankings depend on the chosen centrality, traffic, and flood-risk assumptions.
- Custom dashboard scales are useful for exploration, but final report figures should use saved artifacts.
- Some graph and dashboard artifact names are not perfectly consistent; check file names before final submission.

18. Recommended Demo Order

1. Open the dashboard home page.
2. Show the severity selector and map layers.
3. Open Scenario Explorer and explain severity comparison.
4. Open Model Results and explain the trained model and feature importance.
5. Open Optimization Analysis and compare greedy repair with random repair.
6. Open Safe Rerouting and demonstrate how failed roads are avoided.
7. Open a run detail page to show experiment tracking.

19. How A Beginner Should Read The Code

Read the project in this order:

1. ``README.md``
2. ``risk/flood_risk.py``

3. ``graph/cascade.py``
4. ``run_scenario.py``
5. ``ml/dataset.py``
6. ``ml/train_model.py``
7. ``optimization/greedy_repair.py``
8. ``optimization/compare_strategies.py``
9. ``infra_ai_system/dashboard/models.py``
10. ``infra_ai_system/dashboard/views.py``
11. dashboard templates in ``infra_ai_system/dashboard/templates/dashboard/``

That order follows the real story of the project: data, risk, graph, simulation, learning, optimization, interface.

20. Current Project Facts From Saved Files

The current base graph contains 63010 nodes, 93727 edges, and 62143 road nodes.

Saved Scenario Results

- 0.7 mild: 9668 failed roads in ``data/graphs/pune_after_cascade_weather_0.7.gpickle``; initial failures = 9668; cascade steps = 1; peak step failures = 0
- 1.2 severe: 61992 failed roads in ``data/graphs/pune_after_cascade_weather_1.2.gpickle``
- 1.5 very severe: 62012 failed roads in ``data/graphs/pune_after_cascade_weather_1.5.gpickle``; initial failures = 20854; cascade steps = 7; peak step failures = 24543
- 2.5 extreme: 62078 failed roads in ``data/graphs/pune_after_cascade_weather.gpickle``; initial failures = 34792; cascade steps = 5; peak step failures = 24823

Current Best Model Metrics

- selected model: Random Forest
- ROC-AUC: 0.9999335595185436
- PR-AUC: 0.99999686243022
- F1: 0.9997785389104618
- balanced accuracy: 0.9853087641964438
- train rows: 1089833
- validation rows: 403518
- test rows: 399219

Current Candidate Model Comparison

- Random Forest: validation ROC-AUC 0.9999572557556465, validation PR-AUC 0.99999802803549, validation F1 0.9997908533184606
- Extra Trees: validation ROC-AUC 0.9999509067629255, validation PR-AUC 0.999997735315509, validation F1 0.9997908580050343

- Logistic Regression: validation ROC-AUC 0.9999253375644481, validation PR-AUC 0.9999996556208006, validation F1 0.9997223290985408

Current Greedy vs Random Repair Comparison

- K=0: greedy failed = 61980.0, random failed = 61987.6
- K=500: greedy failed = 61984.8, random failed = 61974.2
- K=2000: greedy failed = 61982.8, random failed = 61982.2
- K=5000: greedy failed = 61973.8, random failed = 61974.0

Current Repair Impact Curve

- K=0: failed = 61984 of 62143, fraction failed = 0.9974413851922179
- K=500: failed = 57181 of 62143, fraction failed = 0.9201519076967639
- K=2000: failed = 51301 of 62143, fraction failed = 0.8255314355599183
- K=5000: failed = 40531 of 62143, fraction failed = 0.652221489146002
- K=10000: failed = 28471 of 62143, fraction failed = 0.4581529697632879

21. File-By-File Code Map

data_ingestion/weather_loader.py

Loads rainfall NetCDF files, previews rainfall variables, clips rainfall data to the Pune bounding box, and plots sample rainfall snapshots.

- Main imports: xarray, matplotlib.pyplot, pathlib
- Function ``load_year(filepath)``, line 8
- Function ``preview_year(filepath)``, line 15
- Function ``clip_to_pune(ds)``, line 31

graph/__init__.py

Project Python source file.

- No top-level functions or classes.

graph/cascade.py

Contains the shared cascade step that spreads failures from already failed road nodes to nearby road nodes using stress, capacity, and overload.

- Main imports: random
- Function ``cascade_step_capacity(G, overload_factor)``, line 4

infra_ai_system/dashboard/__init__.py

Project Python source file.

- No top-level functions or classes.

infra_ai_system/dashboard/admin.py

Configures Django admin screens for research models.

- Main imports: django.contrib, models
- Class `ScenarioConfigAdmin`, line 13
- Class `RunArtifactInline`, line 27
- Class `ModelResultInline`, line 32
- Class `OptimizationResultInline`, line 37
- Class `ExperimentRunAdmin`, line 43
- Class `RunArtifactAdmin`, line 60
- Class `ModelResultAdmin`, line 67
- Class `OptimizationResultAdmin`, line 80

infra_ai_system/dashboard/apps.py

Project Python source file.

- Main imports: django.apps
- Class `DashboardConfig`, line 4

infra_ai_system/dashboard/management/__init__.py

Project Python source file.

- No top-level functions or classes.

infra_ai_system/dashboard/management/commands/__init__.py

Project Python source file.

- No top-level functions or classes.

infra_ai_system/dashboard/management/commands/export_dashboard_artifacts.py

Precomputes enriched map artifacts for faster dashboard page loading.

- Main imports: json, pathlib, django.core.management.base, dashboard
- Class `Command`, line 9
- Method `handle(self)`, line 12 - Method `write\_json(self, path, payload)`, line 46 - Method `sanitize(self, value)`, line 51

infra_ai_system/dashboard/management/commands/ingest_research_data.py

Imports saved research outputs into structured Django database rows.

- Main imports: json, pathlib, pandas, django.core.management.base, dashboard.models
- Class `Command`, line 47
- Method `handle(self)`, line 50 - Method `load\_cascade\_metrics(self, path)`, line 248 - Method `load\_dataset\_summary(self, data\_dir)`, line 264 - Method `ingest\_optimization\_results(self, run, path)`, line 281 - Method `upsert\_artifact(self, run, label, artifact\_kind, relative\_path, file\_format)`, line 311

infra_ai_system/dashboard/migrations/0001_initial.py

Project Python source file.

- Main imports: django.db.models.deletion, django.utils.timezone, django.db
- Class `Migration`, line 8

infra_ai_system/dashboard/migrations/__init__.py

Project Python source file.

- No top-level functions or classes.

infra_ai_system/dashboard/models.py

Defines the database schema for scenarios, experiment runs, artifacts, model results, and optimization results.

- Main imports: pathlib, django.conf, django.db, django.utils, django.utils.text
- Class `TimeStampedModel`, line 9
- Class `ScenarioConfig`, line 17
- Method `\_\_str\_\_(self)`, line 35 - Method `save(self)`, line 38
- Class `ExperimentRun`, line 44
- Method `\_\_str\_\_(self)`, line 86 - Method `save(self)`, line 89
- Class `RunArtifact`, line 95
- Method `\_\_str\_\_(self)`, line 124 - Method `absolute\_path(self)`, line 128
- Class `ModelResult`, line 133
- Method `\_\_str\_\_(self)`, line 164
- Class `OptimizationResult`, line 168
- Method `\_\_str\_\_(self)`, line 185

infra_ai_system/dashboard/tests.py

Tests model behavior, page rendering, custom scale handling, artifact loading, and route search.

- Main imports: pathlib, unittest.mock, django.test, django.urls, models,
- Class `ResearchModelsTest`, line 11
- Method `setUp(self)`, line 12 - Method `test\_slugs\_are\_generated(self)`, line 33 - Method `test\_artifact\_resolves\_repo\_relative\_path(self)`, line 40 - Method `test\_related\_results\_attach\_to\_run(self)`, line 54
- Class `DashboardViewsTest`, line 92

- Method ``setUp(self)``, line 93 - Method ``test_dashboard_page_renders(self)``, line 151 - Method ``test_dashboard_accepts_custom_scale(self)``, line 159 - Method ``test_serialized_roads_for_preset_prefs_exported_artifact(self)``, line 164 - Method ``test_scenario_explorer_page_renders(self)``, line 184 - Method ``test_model_results_page_renders(self)``, line 191 - Method ``test_optimization_analysis_page_renders(self)``, line 198 - Method ``test_route_planner_page_renders(self)``, line 205 - Method ``test_route_search_endpoint_renders_json(self)``, line 211 - Method ``test_run_detail_page_renders(self)``, line 216

infra_ai_system/dashboard/urls.py

Maps dashboard URLs to view functions.

- Main imports: `django.urls`,

infra_ai_system/dashboard/views.py

Loads artifacts, enriches road GeoJSON, simulates custom severities, prepares page contexts, and handles dashboard/routing pages.

- Main imports: `json`, `math`, `pickle`, `random`, `functools`, `pathlib`, `networkx`, `pandas`, `django.db.models`, `django.http`, ...
- Function ``repo_root()``, line 25
- Function ``dashboard_artifact_dir()``, line 29
- Function ``_load_json_cached(path_str)``, line 34
- Function ``load_json(path)``, line 45
- Function ``_load_pickle_cached(path_str)``, line 50
- Function ``load_pickle(path)``, line 58
- Function ``available_severities()``, line 62
- Function ``parse_custom_scale(raw_value)``, line 91
- Function ``tuning_for_scale(scale)``, line 103
- Function ``cascade_step_capacity_local(graph, overload_factor)``, line 111
- Function ``select_severity(severity_slug)``, line 148
- Function ``load_roads_geojson()``, line 154
- Function ``load_base_graph()``, line 158
- Function ``simulate_scale(scale, seed)``, line 163
- Function ``load_failed_road_ids(graph_path)``, line 227
- Function ``load_criticality_map()``, line 239
- Function ``safe_float(value, default)``, line 259
- Function ``percentile_lookup(values_by_id)``, line 270
- Function ``feature_center(feature)``, line 282
- Function ``roads_feature_lookup()``, line 302
- Function ``route_catalog()``, line 316
- Function ``road_display_label(road_id)``, line 340
- Function ``build_graph_attribute_maps(graph)``, line 349
- Function ``critical_threshold(criticality_map, percentile)``, line 377

- Function ``resolve_feature_road_id(feature)``, line 391
- Function ``enrich_roads_geojson(roads_geojson, graph, failed_ids, criticality_map)``, line 415
- Function ``critical_road_summary()``, line 462
- Function ``route_examples(limit)``, line 498
- Function ``search_route_options(query, limit)``, line 517
- Function ``routing_weight(graph, mode)``, line 540
- Function ``build_route_geojson(route_ids)``, line 559
- Function ``route_metrics(graph, route_ids)``, line 569
- Function ``safest_route_result(source_id, target_id, failed_ids, mode)``, line 588
- Function ``summarize_selected_severity(severity, failed_count)``, line 630
- Function ``build_severity_overview()``, line 649
- Function ``cached_severity_overview()``, line 668
- Function ``preset_artifact_path(severity_slug)``, line 672
- Function ``severity_overview_artifact_path()``, line 676
- Function ``serialized_roads_for_preset(severity_slug)``, line 681
- Function ``serialized_roads_for_custom(scale)``, line 705
- Function ``resolve_scale_context(request)``, line 719
- Function ``get_dashboard_state(request)``, line 755
- Function ``dashboard(request)``, line 820
- Function ``scenario_explorer(request)``, line 825
- Function ``model_results(request)``, line 831
- Function ``optimization_analysis(request)``, line 897
- Function ``route_planner(request)``, line 942
- Function ``route_search(request)``, line 1023
- Function ``run_detail(request, slug)``, line 1028

infra_ai_system/infra_ai_system/___init___.py

Project Python source file.

- No top-level functions or classes.

infra_ai_system/infra_ai_system/asgi.py

Project Python source file.

- Main imports: `os`, `django.core.asgi`

infra_ai_system/infra_ai_system/settings.py

Django settings for installed apps, database, templates, static files, and development configuration.

- Main imports: `pathlib`

infra_ai_system/infra_ai_system/urls.py

Root URL router that sends dashboard URLs to the dashboard app and exposes the Django admin.

- Main imports: `django.contrib`, `django.urls`, `django.conf`, `django.conf.urls.static`

infra_ai_system/infra_ai_system/wsgi.py

Project Python source file.

- Main imports: `os`, `django.core.wsgi`

infra_ai_system/manage.py

Django command-line entry point.

- Main imports: `os`, `sys`
- Function ``main()``, line 7

ml/dataset.py

Generates the main tabular ML dataset by running many simulated scenarios and recording road, graph, neighbor, exposure, and failure features.

- Main imports: `pickle`, `pandas`, `random`, `sys`, `functools`, `pathlib`, `networkx`, `graph.cascade`
- Function ``static_graph_features()``, line 34
- Function ``simulate_once(scale, run_index)``, line 66
- Function ``build_dataset()``, line 188

ml/export_graph_to_json.py

Exports a saved graph into JSON for web/static visualization use.

- Main imports: `pickle`, `json`, `pathlib`
- Function ``export()``, line 11

ml/generate_gnn_graphs.py

Creates saved post-cascade graphs at different scales for graph neural-network experiments.

- Main imports: `pickle`, `random`, `pathlib`, `graph.cascade`
- Function ``generate_graph(scale)``, line 15
- Function ``main()``, line 43

ml/generate_temporal_graphs.py

Saves step-by-step graph snapshots for temporal cascade analysis.

- Main imports: `pickle`, `random`, `pathlib`, `graph.cascade`
- Function ``simulate_and_save(scale, run_id)``, line 17
- Function ``main()``, line 58

ml/gnn_dataset.py

Converts one saved graph into a PyTorch Geometric Data object for graph neural-network experiments.

- Main imports: pickle, torch, pathlib, torch_geometric.data
- Function `build\_gnn\_data()`, line 10

ml/gnn_dataset_multi.py

Builds normalized graph datasets across multiple severity scales for generalization experiments.

- Main imports: pickle, numpy, pathlib, sklearn.preprocessing, networkx, torch, torch_geometric.data
- Function `compute\_neighbor\_stats(G)`, line 15
- Function `build\_graph(scale)`, line 40
- Function `load\_all\_graphs()`, line 121

ml/stggn_dataset.py

Builds graph pairs/sequences for spatio-temporal graph neural-network experiments.

- Main imports: pickle, torch, numpy, pathlib, sklearn.preprocessing, torch_geometric.data
- Function `load\_graph(scale)`, line 16
- Function `neighbor\_stats(G, node)`, line 26
- Function `build\_graph\_pair(prev\_G, curr\_G)`, line 53
- Function `build\_sequences()`, line 119

ml/temporal_dataset.py

Generates a temporal dataset where each row describes a road at one cascade step and the target is whether it fails in the next step.

- Main imports: pickle, pandas, random, pathlib, graph.cascade
- Function `simulate\_temporal(scale)`, line 19
- Function `build\_dataset()`, line 112

ml/train_gnn.py

Trains a GraphSAGE model on one graph dataset and evaluates ROC-AUC.

- Main imports: torch, torch.nn, torch.nn.functional, torch_geometric.nn, sklearn.metrics, numpy, ml.gnn_dataset
- Class `GraphSAGE`, line 11
- Method `\_\_init\_\_(self, in\_dim)`, line 12 - Method `forward(self, data)`, line 19
- Function `safe\_normalize(x)`, line 34
- Function `train()`, line 44

ml/train_gnn_generalization.py

Trains GraphSAGE on multiple scenarios and tests it on a held-out scenario.

- Main imports: torch, torch.nn, torch.nn.functional, torch_geometric.nn, sklearn.metrics, numpy, ml.gnn_dataset_multi
- Class `GraphSAGE`, line 11
- Method `\_\_init\_\_(self, in\_dim)`, line 12 - Method `forward(self, x, edge\_index)`, line 18
- Function `train()`, line 26

ml/train_model.py

Trains and compares tabular classifiers, selects the best model by validation PR-AUC, evaluates it on a held-out test split, and saves metrics and feature importance.

- Main imports: json, pathlib, joblib, pandas, sklearn.ensemble, sklearn.linear_model, sklearn.metrics, sklearn.model_selection, sklearn.pipeline, sklearn.preprocessing
- Function `build\_feature\_matrix(df)`, line 30
- Function `grouped\_train\_val\_test\_split(df)`, line 36
- Function `best\_threshold(y\_true, probabilities)`, line 55
- Function `evaluate(y\_true, probabilities, threshold)`, line 71
- Function `candidate\_models()`, line 83
- Function `feature\_importance\_frame(model\_name, model, feature\_columns)`, line 127
- Function `main()`, line 143

ml/train_nn.py

Trains a simple feed-forward neural network on the tabular road-failure dataset.

- Main imports: pandas, torch, torch.nn, pathlib, sklearn.model_selection, sklearn.metrics, sklearn.preprocessing, numpy
- Class `FailureNet`, line 14
- Method `\_\_init\_\_(self, in\_dim)`, line 15 - Method `forward(self, x)`, line 25
- Function `main()`, line 29

ml/train_stggn.py

Trains an experimental spatio-temporal GNN with GraphSAGE layers and a GRU.

- Main imports: torch, torch.nn, numpy, sklearn.metrics, torch_geometric.nn, torch_geometric.utils, stggn_dataset
- Class `STGNN`, line 13
- Method `\_\_init\_\_(self, in\_dim, hidden)`, line 15 - Method `forward(self, data\_seq, node\_idx)`, line 30
- Function `train()`, line 65

ml/train_temporal.py

Trains a temporal neural network to predict future failure for roads that are currently alive.

- Main imports: pandas, torch, torch.nn, sklearn.model_selection, sklearn.preprocessing, sklearn.metrics, pathlib
- Class `TemporalNN`, line 13
- Method `\_\_init\_\_(self, input\_dim)`, line 14 - Method `forward(self, x)`, line 26

- Function ``main()``, line 30

optimization/compare_strategies.py

Compares greedy repair against random repair by reinforcing selected roads and measuring final failed-road counts.

- Main imports: `pickle, random, json, pathlib, optimization.greedy_repair, graph.cascade`
- Function ``load_graph()``, line 19
- Function ``copy_graph(G)``, line 24
- Function ``seed_flood_failures(G)``, line 28
- Function ``reinforce(G, road_ids)``, line 37
- Function ``run_cascade(G, steps)``, line 45
- Function ``count_failed(G)``, line 52
- Function ``experiment(K, strategy)``, line 57
- Function ``main()``, line 85

optimization/greedy_repair.py

Loads critical-road scores and selects the top K roads by a normalized centrality-plus-flood-risk score.

- Main imports: `pickle, pandas, pathlib`
- Function ``load_graph()``, line 11
- Function ``load_scores()``, line 16
- Function ``greedy_select(K, alpha, beta)``, line 20

optimization/optimize_budget.py

Placeholder for future budget optimization logic.

- No top-level functions or classes.

optimization/random_baseline.py

Placeholder for future random-baseline logic.

- No top-level functions or classes.

risk/flood_risk.py

Loads rainfall and road GIS data, samples rainfall near each road, calculates flood-risk scores, and attaches rainfall fields to roads.

- Main imports: `geopandas, xarray, shapely.geometry, pathlib, numpy`
- Function ``load_rainfall_year(year)``, line 20
- Function ``clip_to_pune(ds)``, line 38
- Function ``load_roads()``, line 50
- Function ``sample_rainfall_timeseries(ds, lat, lon)``, line 64
- Function ``compute_flood_risk(series)``, line 78

- Function ``attach_rainfall_to_roads(year)``, line 97

run_scenario.py

Standalone script for running one flood scenario, seeding failures from flood risk, executing cascade steps, and saving result JSON and final graph pickle.

- Main imports: `pickle, json, random, pathlib, graph.cascade`
- Function ``seed_failures_from_flood_risk(G, scale)``, line 46
- Function ``main()``, line 75

visualization/animate_cascade.py

Creates an animated visualization of cascade progression.

- Main imports: `pickle, networkx, matplotlib.pyplot, imageio, pathlib`
- Function ``load_graph()``, line 14
- Function ``simulate_cascade(G, steps)``, line 20
- Function ``main()``, line 49

visualization/cascade_progression.py

Plots cascade progression from saved JSON results.

- Main imports: `json, matplotlib.pyplot, pathlib`
- Function ``main()``, line 11

visualization/compare_scenarios.py

Compares failure outcomes across saved scenario JSON files.

- Main imports: `json, pathlib, matplotlib.pyplot`

visualization/critical_roads.py

Computes or visualizes critical roads based on graph centrality and related attributes.

- Main imports: `pickle, pathlib, pandas, networkx`

visualization/heatmap_failures.py

Creates spatial heatmaps for failed roads.

- Main imports: `pickle, pathlib, geopandas, matplotlib.pyplot, shapely.geometry`

visualization/map_failures.py

Maps failed roads after a cascade.

- Main imports: `pickle, pathlib, geopandas, matplotlib.pyplot`

visualization/plot_cascade.py

Plots cascade step counts from result JSON.

- Main imports: json, pathlib, matplotlib.pyplot

visualization/plot_opt_vs_random.py

Plots greedy-vs-random repair comparison.

- Main imports: json, pathlib, matplotlib.pyplot

visualization/plot_repair_curve.py

Plots repair-budget impact curves.

- Main imports: json, pathlib, matplotlib.pyplot

visualization/repair_impact.py

Measures how different repair budgets reduce failure under a scenario.

- Main imports: pickle, json, random, pathlib, pandas, graph.cascade
- Function `seed\_failures(G, scale)`, line 27
- Function `reset\_graph(G)`, line 50
- Function `run\_for\_k(K, critical\_ids)`, line 65

visualization/resilience_curve.py

Plots resilience or failure fraction curves from saved outputs.

- Main imports: json, matplotlib.pyplot, pathlib
- Function `main()`, line 11

visualization/resilience_metrics.py

Computes simple graph resilience metrics.

- Main imports: pickle, pathlib, networkx

visualization/visualize_cascade.py

Visualizes cascade results on the graph.

- Main imports: pickle, networkx, matplotlib.pyplot, pathlib
- Function `load\_graph()`, line 13
- Function `main()`, line 19

22. Dashboard Template Map

infra_ai_system/dashboard/templates/dashboard/base.html

Shared page shell, navigation, hero/header blocks, and severity/custom-scale form.

infra_ai_system/dashboard/templates/dashboard/dashboard.html

Main dashboard page with scenario selector, severity selector, Leaflet infrastructure map, cascade summary, training snapshot, and optimization snapshot.

infra_ai_system/dashboard/templates/dashboard/model_results.html

Model page with summary cards, model performance chart, candidate comparison, feature importance, benchmark reference, and training runs.

infra_ai_system/dashboard/templates/dashboard/optimization_analysis.html

Optimization page with budget sensitivity chart and greedy-vs-random result tables.

infra_ai_system/dashboard/templates/dashboard/route_planner.html

Interactive safe-rerouting page with road search, map clicks, route comparison, and mode-specific route weighting.

infra_ai_system/dashboard/templates/dashboard/run_detail.html

Experiment-run detail page showing metrics, inputs, artifacts, model results, and optimization rows.

infra_ai_system/dashboard/templates/dashboard/scenario_explorer.html

Severity comparison page with Chart.js visualizations, selected severity metrics, interpretation notes, and critical-road spatial summary.

23. Notebook Map

- `notebooks/01\_prepare\_gis.ipynb`: prepares/clips road and bridge GIS layers and saves final GeoJSON files.
- `notebooks/02\_load\_gis.ipynb`: explores GIS data, assigns traffic/length, joins bridge information, and builds the base graph.
- `notebooks/03\_visualize\_failures.ipynb`: explores and plots failure results after cascade simulations.

24. Common Questions And Answers

Why use a graph?

Roads are connected. A graph gives the project a natural way to represent connectivity and neighbor effects. Without a graph, the project could still show flood risk, but it could not model cascading failures through the network.

Why use random failure?

Flood impact is uncertain. The project uses risk multiplied by severity as a probability. That makes repeated simulations possible and creates datasets for ML.

Why does the ML model score so high?

The labels come from a controlled simulation. The features also include strong graph and neighbor signals. That can make the classification problem easier than a real-world prediction problem. In a final report, present the score as simulation-label performance, not proof of real-world deployment readiness.

What is the most important code file?

There is no single file. The core chain is:

```
risk/flood_risk.py
graph/cascade.py
run_scenario.py
ml/dataset.py
ml/train_model.py
optimization/compare_strategies.py
infra_ai_system/dashboard/views.py
```

What should be improved next?

- better flood-risk resolution
- real traffic data instead of traffic heuristics
- observed failure labels if available
- stronger spatial train/test splitting
- cleaner experiment artifact naming
- more robust route-planning validation
- deployment hardening for Django settings and secrets

25. Final Mental Model

Remember the project as five layers:

1. Data layer: roads, bridges, rainfall, graph files, datasets.
2. Simulation layer: flood risk seeds failures, cascade spreads failures.
3. Learning layer: models learn patterns from simulated failures.
4. Decision layer: optimization ranks or repairs roads under budgets.
5. Interface layer: Django turns the research outputs into pages and maps.

If you understand those five layers, the project becomes much easier to explain.

26. One-Page Beginner Summary

The project studies flood resilience in Pune's road network. It begins with GIS road and bridge data and rainfall files. The code assigns each road a flood-risk score, builds a graph where roads are connected, and simulates how floods can cause initial road failures. A cascade model then spreads failures to nearby roads when stress exceeds capacity. The generated simulations become datasets for machine learning, where models predict road failure from traffic, rainfall, graph, and neighbor features. Optimization scripts compare which roads should be reinforced first under limited budgets. A Django dashboard presents the results with maps, severity controls, model charts, optimization charts, experiment records, and safe rerouting.

That is the whole project in one sentence: it is an end-to-end urban infrastructure resilience prototype that connects geospatial data, rainfall risk, graph cascade simulation, machine learning, optimization, and dashboard visualization.